



Tutorials ▾

References ▾

Exercises ▾

Certificates ▾

Search



Sign In

[HTML](#) [CSS](#) [JAVASCRIPT](#) [SQL](#) [PYTHON](#) [JAVA](#) [PHP](#) [HOW TO](#) [W3.CSS](#) [C](#) [C++](#) [C#](#) [BOOT!](#) >

- Python Tuples ▾
- Python Sets ▾
- Python Dictionaries ▾
- Python If...Else
- Python Match
- Python While Loops
- Python For Loops
- Python Functions
- Python Decorators
- Python Range
- Python Lambda
- Python Arrays
- Python OOP
- Python Classes/Objects
- Python Inheritance
- Python Iterators
- Python Polymorphism
- Python Scope
- Python Modules
- Python Dates
- Python Math
- Python JSON
- Python RegEx
- Python PIP
- Python Try...Except
- Python String Formatting
- Python None
- Python User Input
- Python VirtualEnv

File Handling

- Python File Handling
- Python Read Files
- Python Write/Create Files

Python Polymorphism

[< Previous](#)[Next >](#)

The word "polymorphism" means "many forms", and in programming it refers to methods/functions/operators with the same name that can be executed on many objects or classes.

Function Polymorphism

An example of a Python function that can be used on different objects is the [len\(\)](#) function.

String

For strings [len\(\)](#) returns the number of characters:

Example

```
x = "Hello World!"  
  
print(len(x))
```

[Try it Yourself »](#)[Get your own Python Server](#)

Tuple

For tuples [len\(\)](#) returns the number of items in the tuple:

Example

```
mytuple = ("apple", "banana", "cherry")
```

ADVERTISEMENT

LEARN PYTHON
GET CERTIFIED
CHECK IT OUT



W3schools

COLOR
PICKER



```
print(len(mytuple))
```

[Try it Yourself »](#)

AD

Dictionary

For dictionaries `len()` returns the number of key/value pairs in the dictionary:

Example

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
  
print(len(thisdict))
```

[Try it Yourself »](#)

ADVERTISEMENT

[REMOVE ADS](#)

R

Class Polymorphism

Polymorphism is often used in Class methods, where we can have multiple classes with the same method name.

For example, say we have three classes: `Car`, `Boat`, and `Plane`, and they all have a method called `move()`:

Example

Different classes with the same method:

```
class Car:  
    def __init__(self, brand, model):
```

```
        self.brand = brand
        self.model = model

    def move(self):
        print("Drive!")

class Boat:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def move(self):
        print("Sail!")

class Plane:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def move(self):
        print("Fly!")

car1 = Car("Ford", "Mustang")           #Create a Car
object
boat1 = Boat("Ibiza", "Touring 20")    #Create a Boat
object
plane1 = Plane("Boeing", "747")        #Create a Plane
object

for x in (car1, boat1, plane1):
    x.move()
```

Try it Yourself »

Look at the for loop at the end. Because of polymorphism we can execute the same method for all three classes.

Inheritance Class Polymorphism

What about classes with child classes with the same name? Can we use polymorphism there?

Yes. If we use the example above and make a parent class called **Vehicle**, and make **Car**, **Boat**, **Plane** child classes of **Vehicle**, the child classes inherits the **Vehicle** methods, but can override them:

Example

Create a class called **Vehicle** and make **Car**, **Boat**, **Plane** child classes of **Vehicle**:

```
class Vehicle:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def move(self):
        print("Move!")

class Car(Vehicle):
    pass

class Boat(Vehicle):
    def move(self):
        print("Sail!")

class Plane(Vehicle):
    def move(self):
        print("Fly!")

car1 = Car("Ford", "Mustang")          #Create a Car
object
boat1 = Boat("Ibiza", "Touring 20")    #Create a Boat
object
plane1 = Plane("Boeing", "747")        #Create a Plane
object

for x in (car1, boat1, plane1):
    print(x.brand)
    print(x.model)
    x.move()
```

Try it Yourself »

Child classes inherits the properties and methods from the parent class.

In the example above you can see that the **Car** class is empty, but it inherits **brand**, **model**, and **move()** from **Vehicle**.

The **Boat** and **Plane** classes also inherit **brand**, **model**, and **move()** from **Vehicle**, but they both override the **move()** method.

Because of polymorphism we can execute the same method for all classes.

Exercise ?